

Release Code Review Report

Review date: 2026-05-24

Repository: Sea Lyon Media Manager

Review mode: report-only, no application source changes

Executive Summary

Sea Lyon Media Manager is close to a 1.0 candidate from a functional-code perspective: the local automated suite is broad and currently passes, the Windows packaging workflow is detailed, and several previous release-readiness gaps appear to have been addressed.

However, the project is not yet ready for an unconditional stable 1.0 release. The main blocker is in the release publication flow: the GitHub Actions workflow deploys the update appcast to GitHub Pages immediately after creating a draft release. That can advertise an unapproved or inaccessible release to existing installations before manual smoke testing and final publication.

The other release risks are manageable but should be fixed before 1.0: embedded Last.fm API credentials in packaged builds, incomplete dependency locking, inconsistent local-vs-CI binary resolution, stale release documentation, and a few user-facing correctness issues.

Verification performed during this review:

- `PYTHONDONTWRITEBYTECODE=1 pytest -q -p no:cacheprovider -> 702 passed in 50.11s`
- `PYTHONDONTWRITEBYTECODE=1 pytest --collect-only -q -p no:cacheprovider -> 702 tests collected`
- `.venv/bin/python -m pip check -> No broken requirements found`
- AST parse of `lyon/` with `.venv/bin/python -> no syntax errors`
- Checked for common PDF generators: `pandoc`, `wkhtmltopdf`, and `weasyprint` were not installed
- Generated this report's PDF with the repository's local PySide6-based Markdown-to-PDF renderer

Overall Release Readiness

Recommendation: Conditionally ready after specific fixes.

The application code and test suite look substantially healthier than an early pre-1.0 project, but the current release workflow can expose bad update metadata to users. A stable 1.0 release should not ship until that workflow is corrected and the high-priority release, dependency, and credential issues below are resolved.

Release readiness by area:

Area	Status	Notes
Core functionality	Mostly ready	Automated tests pass; manual Windows smoke testing still required.
Packaging	Needs fixes	CI is robust, but appcast publishing order is unsafe.
Updater	Needs fixes	Release appcast flow is unsafe; version comparison has pre-release edge cases.
Dependencies	Needs fixes	Runtime direct dependencies are pinned, but transitive and build dependencies are not fully locked.
Security posture	Needs fixes	Last.fm API secret is embedded into packaged binaries; DLNA exposure should be clearer and more constrained.
Documentation	Needs cleanup	Version and build docs disagree with code and workflow reality.
Test strategy	Good baseline	702 tests pass, but major release smoke tests remain manual and Windows-specific.

Critical Blockers

CR-1: Appcast is deployed before the draft release is approved or public

- Severity: Critical
- File path: `.github/workflows/windows-build.yml`

- Module/function/component: Tag release workflow, appcast generation and Pages deployment
- Location: Approximately lines 365-413

Description:

On tag builds, the workflow generates `dist/appcast.xml`, creates a draft GitHub Release, then immediately uploads and deploys the appcast to GitHub Pages. The appcast contains URLs such as `https://github.com/${{github.repository}}/releases/download/vX.Y.Z/...`, but those assets are attached to a draft release until a maintainer publishes it.

Why it matters:

Existing installations can see the new appcast before the release has passed manual smoke testing or before GitHub exposes the download assets publicly. Users could be prompted to install a build that is still draft, unapproved, or inaccessible. For an auto-update path, that is a 1.0 release blocker.

How to reproduce or verify:

1. Push a tag that triggers the workflow.
2. Observe that Create draft GitHub Release runs at lines 384-396.
3. Observe that Deploy appcast to GitHub Pages still runs at lines 411-413 while the release is draft.
4. Check the Pages-hosted `appcast.xml`; it advertises the new installer before the release is published.

Suggested fix:

Separate build creation from update publication. Generate the appcast as an artifact, but only deploy it after manual smoke testing and after the GitHub Release is published. Good options include:

- A separate manually approved `publish-appcast` workflow.
- A workflow triggered by the `GitHub release.published` event.
- A protected deployment environment requiring maintainer approval after smoke tests.

High Priority Issues

H-1: Last.fm API secret is embedded into packaged desktop binaries

- Severity: High
- File path: `.github/workflows/windows-build.yml`, `lyon/core/scrobbler.py`
- Module/function/component: Write API secrets workflow step; scrobbler credential loading and request signing
- Location: Workflow lines 232-239; `scrobbler.py` approximately lines 19-26 and 51-54

Description:

The Windows build writes `LYON_LASTFM_API_KEY` and `LYON_LASTFM_API_SECRET` into `lyon\core\_secrets.py` immediately before PyInstaller packaging. The runtime scrobbler module imports those values and uses the secret to sign Last.fm requests.

Why it matters:

A desktop binary is under the user's control. Any secret embedded into the package should be considered extractable. If the Last.fm secret is compromised, third parties can impersonate the application, abuse the API key, and force a credential rotation after release.

How to reproduce or verify:

1. Build with `LYON_LASTFM_API_SECRET` populated.
2. Inspect the PyInstaller output, extracted archive, or bundled Python bytecode for `_secrets.py` content.
3. Confirm that the API secret can be recovered from the packaged application.

Suggested fix:

Do not treat a bundled desktop credential as secret. Prefer one of:

- Use only public client credentials suitable for distribution.
- Proxy signing through a service controlled by the project.
- Let advanced users provide their own Last.fm API credentials.

- If embedding is accepted as a product decision, document it explicitly and remove the "secret" assumption from release controls.

H-2: Release dependency locking is incomplete

- Severity: High
- File path: requirements.txt, requirements-build.txt
- Module/function/component: Dependency configuration
- Location: requirements.txt lines 1-9; requirements-build.txt lines 1-4

Description:

requirements.txt states that transitive dependency pinning is a v1.0 blocker, but only direct runtime dependencies are pinned. requirements-build.txt includes unpinned build/test tooling such as pyinstaller and pytest.

Why it matters:

Stable 1.0 builds need reproducibility. A clean build performed after a transitive dependency or build tool release could behave differently, fail packaging, or ship a subtly different runtime.

How to reproduce or verify:

1. Create two clean virtual environments at different times.
2. Install from requirements-build.txt.
3. Compare pip freeze output and PyInstaller build results.

Suggested fix:

Generate locked runtime and build requirement files with hashes, for example with pip-tools or an equivalent process. Pin PyInstaller, pytest, and all transitive dependencies used in the release build.

H-3: Local Windows build still resolves the latest fpcalc release dynamically

- Severity: High
- File path: scripts/build-windows.ps1
- Module/function/component: Get-LatestFpcalcRelease
- Location: Approximately lines 113-132 and the call site around lines 231-232

Description:

The local Windows build script resolves <https://api.github.com/repos/acoustid/chromaprint/releases/latest> and downloads the latest matching fpcalc asset. CI appears to use a pinned version, but the local release path can produce a different binary set.

Why it matters:

Release artifacts built locally can diverge from CI artifacts. A new upstream Chromaprint release could introduce a regression, a renamed asset, or behavior differences without any source change in this repository.

How to reproduce or verify:

1. Run the local build script after a new Chromaprint release.
2. Compare the bundled fpcalc.exe version with the version used in CI.

Suggested fix:

Make the local script use the same pinned Chromaprint/fpcalc version and URL as CI. Add a SHA256 verification step or a checked-in manifest for the fpcalc artifact.

H-4: Public version and release documentation is stale

- Severity: High
- File path: README.md, lyon/__init__.py, docs/BUILD.md, .github/workflows/windows-build.yml
- Module/function/component: Public documentation and release metadata
- Location: README.md approximately lines 15-16; lyon/__init__.py line 2; docs/BUILD.md approximately lines 100-119; workflow env PYTEST_FLOOR

Description:

The README identifies the current version as 0.8.0, while the package version is 0.9.0-rc1. Build documentation

still describes tag-triggered releases as planned, while the workflow already has tag triggers. The build docs also mention a 645-test floor while the workflow uses PYTEST_FLOOR: "680" and the current collection is 702 tests.

Why it matters:

For 1.0, public documentation, package metadata, release notes, and installer metadata must agree. Mismatched version information causes support confusion and makes release verification harder.

How to reproduce or verify:

1. Compare the README version text with `lyon/__init__.py`.
2. Compare docs/BUILD.md release workflow notes with `.github/workflows/windows-build.yml`.
3. Compare the documented test floor with workflow configuration and `pytest --collect-only`.

Suggested fix:

Update documentation as part of the release checklist. Keep one authoritative version source and ensure README, installer, build docs, and release notes are updated in the same release PR.

H-5: 1.0 release cannot be fully certified from this environment alone

- Severity: High
- File path: docs/RELEASE_CHECKLIST.md, build/lyon.iss, .github/workflows/windows-build.yml
- Module/function/component: Manual release validation
- Location: Release checklist and Windows packaging configuration

Description:

The automated Python test suite passes locally, but this review environment did not run the built Windows installer or exercise packaged VLC, ffmpeg, fpcalc, libdiscid, CD hardware, Windows file associations, firewall behavior, or update installation on Windows 10/11.

Why it matters:

Sea Lyon is a Windows desktop media application with native binaries and installer behavior. Stable 1.0 requires installer and runtime validation on the target platform, not only unit tests on a development machine.

How to reproduce or verify:

Run the manual QA checklist in this report and in docs/RELEASE_CHECKLIST.md against a fresh Windows VM and at least one real Windows desktop with audio output.

Suggested fix:

Make successful Windows installer smoke testing a release gate. Capture OS version, installer checksum, app version, logs, and pass/fail notes for the release record.

Medium Priority Issues

M-1: Drag-and-drop can report unsupported files as successfully added

- Severity: Medium
- File path: `lyon/ui/main_window.py`, `lyon/core/library.py`
- Module/function/component: `MainWindow.dropEvent`, library file indexing
- Location: `main_window.py` lines 1707-1748; `library.py` supported-extension/index result handling

Description:

The UI drag/drop allowlist includes extensions such as `.ape`, `.alac`, and `.mka`. The library's supported extension set does not appear to accept all of those formats. `dropEvent` calls `self.library.add_file(f)` but ignores the returned result, commits, refreshes, and shows a success toast based only on the number of dropped files.

Why it matters:

Users can receive a false "Added N files" success message even when the library skipped or rejected the file. This is especially visible during first-run library import and undermines trust in the catalog.

How to reproduce or verify:

1. Drag a local `.ape`, `.alac`, or otherwise unsupported file into the main window.
2. Observe the success toast.

3. Check whether the file actually appears in the library.

Suggested fix:

Align the drag/drop allowlist with `Library.SUPPORTED_EXTS`, or count `IndexResult` statuses from `add_file()` and show accurate added/skipped/error feedback.

M-2: Updater version comparison ignores pre-release semantics

- Severity: Medium
- File path: `lyon/core/updater.py`
- Module/function/component: `_parse_version`
- Location: Approximately lines 83-97

Description:

The updater strips pre-release and build metadata before comparing version tuples. As a result, `1.0.0-rc1` and `1.0.0` compare as equal.

Why it matters:

Users running a future release candidate such as `1.1.0-rc1` may not be offered the final `1.1.0` update. For the current `0.9.0-rc1` to `1.0.0` path this is not blocking, but it is a release-channel correctness issue.

How to reproduce or verify:

Add or inspect tests comparing an installed pre-release to the same final version. The current parser treats the numeric components as identical.

Suggested fix:

Use a version parser that understands pre-release ordering, such as `packaging.version.Version`, and add tests for rc-to-final update behavior.

M-3: Appcast generation in CI does not preserve previously published items

- Severity: Medium
- File path: `scripts/generate-appcast.py`, `.github/workflows/windows-build.yml`
- Module/function/component: Appcast generation
- Location: `generate-appcast.py` approximately lines 110-121; workflow lines 365-380

Description:

The appcast generator can preserve existing items only if the output file already exists. The workflow writes a new `dist\appcast.xml` without fetching the current Pages-hosted appcast first.

Why it matters:

Each release may replace the appcast history with a single latest item. That may be acceptable, but it contradicts the script's preservation behavior and removes release metadata that update clients or diagnostics may rely on later.

How to reproduce or verify:

1. Inspect `dist/appcast.xml` after a tag build.
2. Confirm whether older appcast items are present.
3. Check whether the workflow ever downloads the existing Pages appcast before generation.

Suggested fix:

Either fetch the existing Pages appcast before generation or document and test that the product intentionally serves latest-only update metadata.

M-4: DLNA server exposes the library to all private/link-local clients when enabled

- Severity: Medium
- File path: `lyon/core/dlna_server.py`
- Module/function/component: `DLNARequestHandler._client_allowed`, `_is_allowed_client`, `LyonDLNAServer.start`
- Location: Lines 742-748 and 1153-1158; server bind setup around lines 104-123

Description:

The DLNA server allows loopback, private, and link-local clients. That is reasonable for DLNA, but it means any device on a private network, guest LAN, VPN adapter, or local virtual network that can reach the server may browse and stream the exposed library while DLNA is enabled.

Why it matters:

This is not necessarily a vulnerability because DLNA is intentionally local-network sharing, but it is a sensitive default for a media library. Users may not understand the scope of "local network" on machines with VPNs or multiple adapters.

How to reproduce or verify:

1. Enable DLNA.
2. From another private-network client, request the device description, browse endpoint, or media stream URL.
3. Confirm access is allowed without authentication.

Suggested fix:

Before 1.0, make the sharing boundary explicit in the UI and release docs. Consider binding to a selected interface, showing the active bind address, adding an "allow only this subnet/interface" option, or requiring a more explicit confirmation before enabling DLNA.

M-5: Update failures may retry on every startup

- Severity: Medium
- File path: `lyon/ui/main_window.py`
- Module/function/component: `_start_update_check`, `_on_update_check_failed`
- Location: Approximately lines 1582-1658

Description:

The update check is throttled by `last_update_check_ts` on success/no-update paths, but failure handling does not appear to update that timestamp. If the update endpoint is unavailable, the application can retry every startup instead of backing off.

Why it matters:

Repeated failed network checks can slow startup, create noisy logs, and make outages look like an application issue.

How to reproduce or verify:

1. Configure an unreachable appcast URL.
2. Start the application repeatedly.
3. Observe whether every startup attempts the update request.

Suggested fix:

Record a failed-check timestamp and apply a shorter but explicit retry backoff, for example a few hours, while preserving immediate manual "Check for updates" behavior.

Low Priority Issues

L-1: Build and release docs contain stale future-tense guidance

- Severity: Low
- File path: `docs/BUILD.md`
- Module/function/component: Release documentation
- Location: Approximately lines 100-119

Description:

Some build documentation describes tag-triggered release behavior as planned and documents older test-floor values. This overlaps with H-4 but is listed separately as documentation cleanup.

Suggested fix:

Refresh docs/BUILD.md for the current workflow, current test floor, current artifact list, and 1.0 release process.

L-2: Historical review/planning documents may confuse 1.0 status

- Severity: Low
- File path: FEATURE_AND_CODE_REVIEW.md, IMPLEMENTATION_PLAN_1.0.md, REVIEW_NEXT_STEPS.md
- Module/function/component: Repository documentation
- Location: Repository root

Description:

The repository contains previous review and planning artifacts. Some items appear to describe gaps that have since been fixed. They are not harmful as historical records, but they can confuse maintainers preparing 1.0 if they are read as current status.

Removal classification:

Keep but clean up, archive, or clearly mark as historical before the 1.0 release. Do not delete without confirming whether maintainers still use them as project records.

Suggested fix:

Add an "historical" note to old review files, move them under an archive directory, or replace them with a single current release-readiness tracker.

L-3: No repository-level lint or type-check gate is visible

- Severity: Low
- File path: Repository configuration
- Module/function/component: Tooling configuration
- Location: No pyproject.toml, setup.cfg, tox.ini, Ruff, Black, mypy, Pyright, or flake8 configuration was found during review

Description:

The project has strong unit coverage but does not appear to have a configured lint/type gate. For a large desktop app, this increases the chance of style drift, unused code, accidental broad exception handling, and type-related regressions.

Suggested fix:

Add a conservative lint gate after 1.0 stabilization or as a release hardening task. Start with low-risk checks such as import sorting, syntax/unused import checks, and gradually add stricter typing for core modules.

L-4: Some modules are large enough to slow maintenance

- Severity: Low
- File path: lyon/ui/library_view.py, lyon/ui/main_window.py, lyon/ui/knowledge_base_dialog.py, lyon/ui/video_player_view.py, lyon/core/library.py, lyon/core/metadata.py, lyon/core/dlna_server.py, lyon/core/ripper.py
- Module/function/component: Large UI and core modules
- Location: Whole-file concern

Description:

Several files exceed 1,200 lines, and library_view.py exceeds 2,300 lines. This is not a release blocker by itself, but it makes future bug fixes and review harder.

Suggested fix:

Do not refactor immediately before 1.0 unless a specific bug requires it. After 1.0, split by responsibility: widgets, actions, models/adapters, dialogs, transport/server logic, and formatting helpers.

Bugs and Functional Risks

Severity	Finding	Location	Release impact
Critical	Appcast published before release approval	.github/workflows/windows-build.yml lines 365-413	Can advertise broken/unapproved updates to existing

			installs.
High	Embedded Last.fm secret	Workflow lines 232-239; lyon/core/scrobbler.py	Credentials are extractable from desktop binaries.
High	Non-reproducible dependency graph	requirements*.txt	Builds can change without source changes.
High	Local build downloads latest fpcalc	scripts/build-windows.ps1	Local and CI release artifacts can diverge.
Medium	Drag/drop false success	lyon/ui/main_window.py lines 1707-1748	Users may think files were imported when they were skipped.
Medium	Pre-release update comparison	lyon/core/updater.py lines 83-97	Future rc-to-final updates can be missed.
Medium	DLNA broad local-network access	lyon/core/dlna_server.py lines 742-748, 1153-1158	Library may be reachable from more adapters/clients than users expect.
Medium	Update failure retry behavior	lyon/ui/main_window.py lines 1582-1658	Startup may repeatedly hit unavailable update endpoints.

Previously noted risk areas that appear improved:

- Podcast feed fetches enforce a size cap before parsing.
- Settings cache access is protected by a lock.
- Settings.corrupt_backup_path is an explicit dataclass field.
- Library path move handling appears to avoid holding a database lock while re-indexing.
- Main-window tab activation bounds checks are present.
- DLNA SOAP parsing uses defusedxml and body-size limits.

Stale / Dead / Orphaned Code

Item	Classification	Evidence	Recommendation
FEATURE_AND_CODE_REVIEW.md	Should be kept but cleaned up	Historical review artifact in repository root.	Mark as historical or archive after this report replaces its current-readiness role.
IMPLEMENTATION_PLAN_1.0.md	Should be kept but cleaned up	Some plan items appear already complete.	Convert to a current release checklist or archive.
REVIEW_NEXT_STEPS.md	Needs verification before removal	Mentions issues that appear partly fixed.	Reconcile against this report before removing.
Dynamic latest-fpcalc local build path	Safe to remove after replacement	scripts/build-windows.ps1 resolves latest Chromaprint release.	Replace with pinned artifact logic, then remove latest-resolution helper.
Appcast draft-publication path	Safe to remove after replacement	Pages deployment runs on tag build immediately after draft release creation.	Move to post-publication workflow; remove tag-build Pages deploy step.
Old release documentation text	Safe to update, not delete	README and build docs disagree with current code/workflow.	Update docs in release PR.

No tracked .DS_Store, .pytest_cache, or __pycache__ files were found by git ls-files during this review.

Code Quality Findings

Large coordinator classes

MainWindow, LibraryView, VideoPlayerView, and several core modules own many responsibilities. This is understandable for a desktop app approaching 1.0, but it makes release fixes harder to review.

Recommendation:

Avoid broad refactors before 1.0. For post-1.0 maintenance, split UI files around stable boundaries: toolbar/actions, scan/import orchestration, dialogs, model adapters, playback controls, and device/server concerns.

Release workflow coupling

The Windows workflow handles tests, native dependency download, packaging, installer verification, release creation, appcast generation, and Pages deployment in one job.

Recommendation:

Separate release build, manual approval, release publication, and appcast publication. This improves auditability and prevents the critical appcast issue from recurring.

Configuration drift

Version strings, test floors, release-channel behavior, and binary versions exist in multiple places.

Recommendation:

Create a release metadata checklist or single source of truth for version, test floor, native binary versions, and appcast URL. The project does not need a heavy release system, but it does need one reliable place to verify these values.

Error reporting quality

The app has user-facing toasts and logs, but some paths still collapse success and partial failure into a single success message, notably drag/drop imports.

Recommendation:

For user-initiated file operations, report added/skipped/error counts. Keep raw details in diagnostics logs and expose concise user-facing messages.

Dependency intent is documented but not enforced

requirements.txt explicitly acknowledges transitive pinning as a blocker. That is good, but the release process does not yet enforce it.

Recommendation:

Add a generated lockfile and a CI check that fails if release lockfiles are out of date.

Testing Gaps

Current state:

- 702 tests pass locally.
- Test collection is healthy.
- pip check reports no broken installed requirements.
- The suite covers many core and UI-adjacent behaviors.

Remaining gaps before 1.0:

Area	Gap	Recommended test
Release workflow	Appcast publication order is not tested.	Add a release-process check or split workflow so unsafe ordering cannot occur.
Update flow	Pre-release comparison edge cases.	Test 1.0.0-rc1 installed vs 1.0.0 available.
Drag/drop import	Partial success and unsupported files.	Test that dropped unsupported files are counted as skipped, not added.
Packaged app	Unit tests do not prove packaged VLC/ffmpeg/fpcalc work on Windows.	Installer smoke test on Windows 10 and 11.
DLNA	Network exposure across adapters is hard to	Manual LAN/VPN/guest-network validation.

	infer from unit tests.	
CD ripping	Hardware and drive behavior cannot be fully covered in CI.	Manual rip and AccurateRip/CTDB verification using real hardware.
Updater	End-to-end install/update behavior.	Publish a private test appcast and update from an older packaged build.
Settings upgrade	Existing user profile migration.	Run app against a copied pre-1.0 settings/library directory.

Minimum automated checklist before 1.0:

- Run the full test suite with bytecode/cache disabled for a clean signal.
- Run test collection and ensure the count is above the configured workflow floor.
- Run `pip check`.
- Build the PyInstaller package in CI.
- Verify packaged imports and native binary runtime checks.
- Generate installer and zip artifacts.
- Verify SHA256 manifest generation.

Minimum manual checklist before 1.0:

- Fresh install on Windows 10.
- Fresh install on Windows 11.
- Upgrade install over the previous public build.
- Launch from Start Menu and desktop shortcut.
- Import a small library and a large library.
- Play audio, video, radio, podcasts, and queue transitions.
- Rip a CD on real hardware.
- Enable and disable DLNA.
- Run update check against a test appcast.
- Uninstall and confirm user data handling matches docs.

Dependency and Security Findings

Dependency findings

- Runtime direct dependencies are pinned.
- Transitive dependencies are not locked.
- Build dependencies include unpinned tooling.
- `pip check` passed in the current environment.
- No dependency vulnerability scanner was available locally (`pip-audit` and `safety` were not installed).

Suggested fix:

Before 1.0, generate locked, hash-verified dependency files for both runtime and build environments. Run a dependency vulnerability scanner in CI and record the result in the release checklist.

Credential and secret handling

No plaintext credentials were found in the inspected tracked source files, but CI writes Last.fm credentials into a generated source file for packaging. That generated file is intentionally not checked in, but the resulting binary can still disclose the secret.

Suggested fix:

Resolve H-1 before stable release.

File handling

The codebase contains many file-system features: library scanning, metadata writing, downloads, podcasts, CD

ripping, generated reports, and installers. Notable positives include path containment checks in DLNA media serving and capped podcast feed reads.

Remaining risk:

User-facing file operations should consistently report partial failures and skipped files. Drag/drop import currently appears to overstate success.

Network handling

Network-facing features include update checks, podcasts, radio streams, YouTube integration, Last.fm, MusicBrainz/cover art style metadata flows, and DLNA.

Remaining risks:

- DLNA exposure should be clearer and more constrained.
- Failed updater checks should use an explicit backoff.
- Release appcast publishing must be gated.

Shell/process execution

Build scripts and media tooling invoke external binaries. The inspected Python subprocess usage in appcast generation uses argument lists rather than shell interpolation. Build scripts download native binaries and should keep using pinned URLs and checksum verification for all artifacts.

Platform-Specific Risks

Windows packaging

The project is Windows-first for release packaging. Inno Setup configuration appears substantial, with app metadata, uninstall behavior, license/notice files, and shortcuts. The remaining risks are not obvious syntax issues; they are end-to-end validation risks.

Required validation:

- Installer runs without SmartScreen or signing surprises for the intended distribution path.
- App starts from installed location.
- Native bundled binaries are found from the installed layout.
- VLC playback works on a fresh Windows system.
- ffmpeg-based features work.
- fpcalc fingerprinting works.
- libdiscid/CD features work on hardware where supported.

macOS/Linux development environments

Local unit tests passed in this environment, but passing tests outside Windows does not certify Windows packaging or media-device behavior.

Multi-adapter networks

DLNA behavior should be tested with Wi-Fi, Ethernet, VPN, and guest/private networks. The current private/link-local allow policy can include more clients than users expect.

Update distribution

The appcast must never advertise a release before that release is public and smoke-tested. This is the most important platform/release-specific risk.

Manual QA Checklist

Use this checklist against the final 1.0 release candidate artifacts, not against a development checkout.

Install, launch, and first run

- Install on a clean Windows 10 machine.
- Install on a clean Windows 11 machine.
- Launch from Start Menu.
- Launch from desktop shortcut if enabled.

- Confirm version shown in the app matches the installer, README, release notes, and GitHub tag.
- Confirm first-run defaults are sensible.
- Confirm no startup crash with an empty music library.
- Confirm diagnostic logs are created in the documented location.

Library and file management

- Add a single supported audio file.
- Add a folder containing supported and unsupported files.
- Drag/drop supported files.
- Drag/drop unsupported files and verify accurate skipped/error messaging.
- Scan a large nested library.
- Remove a library path.
- Move or rename a scanned file and verify the app recovers or re-indexes cleanly.
- Verify duplicate handling.
- Verify metadata display for files with missing, partial, and unusual tags.

Playback

- Play MP3, FLAC, AAC/M4A, OGG/Opus, WAV, and WMA if supported.
- Play video formats claimed by the UI.
- Verify play, pause, seek, next, previous, shuffle, repeat, and queue behavior.
- Verify volume, mute, equalizer, ReplayGain, and cross-feature interactions.
- Verify behavior when the playing file is deleted or moved.
- Verify output device changes if the app supports them.

Metadata and enrichment

- Edit common metadata fields.
- Save metadata and reopen the app.
- Test cover art lookup and local cover behavior.
- Test lyrics lookup/display if enabled.
- Test fingerprinting and AcoustID flows when fpcalc is bundled.
- Verify behavior with network unavailable.

CD ripping

- Detect an inserted audio CD.
- Read disc metadata.
- Rip to each supported output mode.
- Verify progress, cancellation, and error messages.
- Verify AccurateRip/CTDB or equivalent verification paths.
- Verify behavior with no drive, unsupported drive, and read errors.

Podcasts, radio, and online media

- Add a valid podcast feed.
- Add an invalid podcast feed.
- Add an oversized or unreachable feed.
- Play and download an episode.
- Add and play radio streams.
- Test YouTube search/download gating and failure handling.
- Confirm network timeouts show useful errors.

DLNA

- Enable DLNA.
- Confirm the UI warns clearly about local network sharing.
- Browse from a trusted LAN client.
- Try from a VPN/private adapter if available.
- Disable DLNA and confirm the server stops.
- Restart the app and confirm DLNA setting persistence matches product intent.

Updater

- Configure a private test appcast for an older packaged build.
- Confirm update detection.
- Confirm download URL opens or downloads correctly.
- Confirm no update is shown for equal/newer installed versions.
- Confirm rc-to-final behavior.
- Confirm failed checks back off and do not nag every startup.

Installer, upgrade, and uninstall

- Upgrade from the previous public version.
- Verify settings and library survive upgrade.
- Verify native binaries are replaced correctly.
- Uninstall and confirm app files are removed.
- Confirm user data retention/deletion behavior matches documentation.
- Reinstall after uninstall.

Accessibility and UI

- Test high-DPI scaling.
- Test keyboard navigation for major actions.
- Test narrow and large window sizes.
- Test dark/light theme behavior if applicable.
- Confirm long paths, long artist/album names, and missing artwork do not break layouts.

Recommended Fix Order

1. Fix the release workflow so appcast deployment happens only after release approval and publication.
2. Decide and implement the Last.fm credential strategy for packaged binaries.
3. Lock runtime and build dependencies, including transitive dependencies and build tools.
4. Pin fpcalc resolution in the local Windows build script and align it with CI.
5. Update README, build docs, release docs, version metadata, and test-floor documentation.
6. Fix drag/drop import feedback so skipped files are reported accurately.
7. Fix updater pre-release comparison semantics.
8. Decide whether appcast history should be preserved; update workflow or docs accordingly.
9. Clarify and optionally narrow DLNA exposure controls.
10. Add updater failure backoff.
11. Run the full Windows release manual QA checklist.
12. Archive or mark stale planning/review documents as historical.

1.0 Release Gate Checklist

The project should not publish 1.0 until all required gates below pass.

Required gates

- ☐ Appcast publication is separated from draft release creation.
- ☐ Draft release is manually smoke-tested before public update metadata is deployed.
- ☐ Last.fm packaged credential strategy is resolved.
- ☐ Runtime dependency lockfile is generated and used.
- ☐ Build dependency lockfile is generated and used.
- ☐ Local and CI Windows builds use the same pinned native binary versions.
- ☐ README, docs, installer metadata, package version, and release notes agree.
- ☐ Full automated test suite passes.
- ☐ Test count meets or exceeds the configured workflow floor.
- ☐ `pip check` passes.
- ☐ Dependency vulnerability scan is run and reviewed.
- ☐ Windows installer smoke test passes on Windows 10.
- ☐ Windows installer smoke test passes on Windows 11.
- ☐ Upgrade from previous public version passes.
- ☐ Updater test appcast path passes.
- ☐ DLNA sharing behavior is documented and manually verified.
- ☐ CD ripping is manually verified on real hardware.
- ☐ Release artifacts have SHA256 checksums.
- ☐ Release notes include known limitations and upgrade notes.

Optional but recommended gates

- ☐ Add conservative lint/import checks.
- ☐ Add rc-to-final updater tests.
- ☐ Add drag/drop partial-success tests.
- ☐ Add a release workflow assertion or separate workflow preventing draft appcast publication.
- ☐ Archive stale planning/review docs.

Final Recommendation

Sea Lyon Media Manager should be treated as conditionally ready after specific fixes, not ready for an unconditional public 1.0 release today.

The automated test suite is in good shape, and the application appears close to a stable candidate. The project should proceed toward 1.0 after correcting the critical appcast publication workflow and resolving the high-priority release reproducibility, credential, and documentation issues.

The minimum bar for 1.0 should be:

1. No draft or untested release can be advertised through the appcast.
2. Packaged builds are reproducible from locked dependencies and pinned native binaries.
3. Public documentation and version metadata agree.
4. Windows installer, updater, playback, library import, DLNA, and CD ripping smoke tests pass on release artifacts.
5. Any remaining known limitations are documented in release notes.